

Coinductive invertibility in higher categories

Ioannis Markakis

University of Cambridge

Higher Structures, Aberdeen

9 September 2026

Why higher categories?

Many mathematical structures are not captured fully by ordinary categories, because their composition is weak and their natural notion of equivalence is higher dimensional.

Why higher categories?

Many mathematical structures are not captured fully by ordinary categories, because their composition is weak and their natural notion of equivalence is higher dimensional.

- Each topological space X gives rise to an ∞ -groupoid $\Pi_\infty(X)$.
- Strict n -categories naturally form an $(n+1)$ -category.
- Manifolds with cobordisms assemble into (∞, n) -categories.

There are various homotopical models of (∞, ∞) -categories built from iterated enrichment or Θ -spaces.

Why higher categories?

Many mathematical structures are not captured fully by ordinary categories, because their composition is weak and their natural notion of equivalence is higher dimensional.

- Each topological space X gives rise to an ∞ -groupoid $\Pi_\infty(X)$.
- Strict n -categories naturally form an $(n+1)$ -category.
- Manifolds with cobordisms assemble into (∞, n) -categories.

There are various homotopical models of (∞, ∞) -categories built from iterated enrichment or Θ -spaces.

Programme

Our goal is to develop an algebraic (set-based) structure modelling (∞, ∞) -categories. Our candidate are the ω -categories of Batanin [3] and Leinster [13].

When is a cell invertible?

Invertibility allows us to define equivalences within and between higher categories.

When is a cell invertible?

Invertibility allows us to define equivalences within and between higher categories.

- A morphism $f: x \rightarrow y$ in an 1-category is invertible if there exist $f^\dagger, f^\rceil: y \rightarrow x$ such that

$$f \circ f^\dagger = 1(x) \qquad 1(y) = f^\rceil \circ f$$

When is a cell invertible?

Invertibility allows us to define equivalences within and between higher categories.

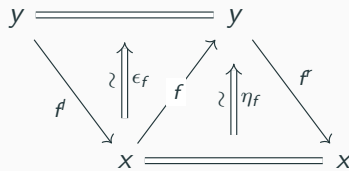
- A morphism $f: x \rightarrow y$ in an 1-category is invertible if there exist $f^\dagger, f^\rceil: y \rightarrow x$ such that

$$f \circ f^\dagger = 1(x) \quad 1(y) = f^\rceil \circ f$$

- A morphism $f: x \rightarrow y$ in an $(n+1)$ -category is invertible if there exist $f^\dagger, f^\rceil: y \rightarrow x$ and invertible

$$\epsilon_f: f \circ f^\dagger \Rightarrow 1(x) \quad \eta_f: 1(y) \Rightarrow f^\rceil \circ f$$

in the n -categories $\text{Hom}(x, x)$ and $\text{Hom}(y, y)$.



When is a cell invertible?

Invertibility allows us to define equivalences within and between higher categories.

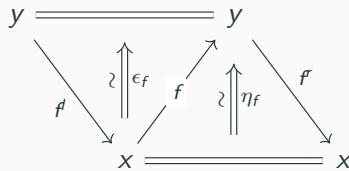
- A morphism $f: x \rightarrow y$ in an 1-category is invertible if there exist $f^\dagger, f^\rceil: y \rightarrow x$ such that

$$f \circ f^\dagger = 1(x) \quad 1(y) = f^\rceil \circ f$$

- A morphism $f: x \rightarrow y$ in an $(n+1)$ -category is invertible if there exist $f^\dagger, f^\rceil: y \rightarrow x$ and invertible

$$\epsilon_f: f \circ f^\dagger \Rightarrow 1(x) \quad \eta_f: 1(y) \Rightarrow f^\rceil \circ f$$

in the n -categories $\text{Hom}(x, x)$ and $\text{Hom}(y, y)$.



Definition

For ω -categories this definition is made **coinductive**: invertibility of a morphism is witnessed by an infinite tower of invertible higher morphisms.

Objectives of this talk

1. What is a weak ω -category?
2. What can we freely generate ω -categories from?
3. Can we make some of the generating data as invertible?

Objectives of this talk

1. What is a weak ω -category?
 - Batanin [3] and Leinster [13] define them as using globular operads
 - Grothendieck [11] and Maltsiniotis [14] define them using limit sketches
 - Finster and Mimram [10] define them using a dependent type theory
2. What can we freely generate ω -categories from?
3. Can we make some of the generating data as invertible?

Objectives of this talk

1. What is a weak ω -category?
 - Batanin [3] and Leinster [13] define them as using globular operads
 - Grothendieck [11] and Maltsiniotis [14] define them using limit sketches
 - Finster and Mimram [10] define them using a dependent type theory
2. What can we freely generate ω -categories from?
 - Globular sets is an easy answer, but not the most general one
 - Batanin [2] and Burroni [8] use computads (a.k.a. polygraphs)
 - Dean, Finster, M., Reutter and Vicary [9] introduce an inductive presentation
 ω -categories freely generated from a computad
3. Can we make some of the generating data as invertible?

Objectives of this talk

1. What is a weak ω -category?
 - Batanin [3] and Leinster [13] define them as using globular operads
 - Grothendieck [11] and Maltsiniotis [14] define them using limit sketches
 - Finster and Mimram [10] define them using a dependent type theory
2. What can we freely generate ω -categories from?
 - Globular sets is an easy answer, but not the most general one
 - Batanin [2] and Burroni [8] use computads (a.k.a. polygraphs)
 - Dean, Finster, M., Reutter and Vicary [9] introduce an inductive presentation ω -categories freely generated from a computad
3. Can we make some of the generating data as invertible?
 - Benjamin, Champin, M. [4] generalise computads to allow inverting generators

Objectives of this talk

1. What is a weak ω -category?
 - Batanin [3] and Leinster [13] define them as using globular operads
 - Grothendieck [11] and Maltsiniotis [14] define them using limit sketches
 - Finster and Mimram [10] define them using a dependent type theory
2. What can we freely generate ω -categories from?
 - Globular sets is an easy answer, but not the most general one
 - Batanin [2] and Burroni [8] use computads (a.k.a. polygraphs)
 - Dean, Finster, M., Reutter and Vicary [9] introduce an inductive presentation ω -categories freely generated from a computad
3. Can we make some of the generating data as invertible?
 - Benjamin, Champin, M. [4] generalise computads to allow inverting generators

Remark

All definitions of ω -categories have been shown equivalent by Ara [1], Bourke [7], Benjamin, Finster, Mimram [6], and Benjamin, M., Sarti [5].

Definition

Strict 0-categories are sets.

Strict $(n + 1)$ -categories are categories enriched in strict n -categories.

The category of strict ω -categories is the limit of the categories of strict n -categories.

Strict higher categories i

Definition

Strict 0-categories are sets.

Strict $(n + 1)$ -categories are categories enriched in strict n -categories.

The category of strict ω -categories is the limit of the categories of strict n -categories.

Pros

- They admit a simple inductive definition.
- Their homotopy theory is understood [12].

Cons

- Most examples of higher categories are not strict.
- Not all tricategories are equivalent to strict ones.

Definition

The category $\mathbb{G}$ of globes is the category generated by the following graph subject to the globularity relations:

$$0 \begin{array}{c} \xrightarrow{\sigma_0} \\ \xleftarrow{\tau_0} \end{array} 1 \begin{array}{c} \xrightarrow{\sigma_1} \\ \xleftarrow{\tau_1} \end{array} 2 \begin{array}{c} \xrightarrow{\sigma_2} \\ \xleftarrow{\tau_2} \end{array} \dots$$

$$\sigma_{n+1} \circ \sigma_n = \tau_{n+1} \circ \sigma_n$$

$$\sigma_{n+1} \circ \tau_n = \tau_{n+1} \circ \tau_n$$

The category $\mathbf{Glob}$ of globular sets is the presheaf category $[\mathbb{G}^{\text{op}}, \mathbf{Set}]$.

Globular sets

Definition

The category $\mathbb{G}$ of globes is the category generated by the following graph subject to the globularity relations:

$$0 \begin{array}{c} \xrightarrow{\sigma_0} \\ \xleftarrow{\tau_0} \end{array} 1 \begin{array}{c} \xrightarrow{\sigma_1} \\ \xleftarrow{\tau_1} \end{array} 2 \begin{array}{c} \xrightarrow{\sigma_2} \\ \xleftarrow{\tau_2} \end{array} \dots$$

$$\sigma_{n+1} \circ \sigma_n = \tau_{n+1} \circ \sigma_n$$

$$\sigma_{n+1} \circ \tau_n = \tau_{n+1} \circ \tau_n$$

The category $\mathbf{Glob}$ of globular sets is the presheaf category $[\mathbb{G}^{\text{op}}, \mathbf{Set}]$.

Example

The disks $\mathbb{D}^n = \mathbb{G}(-, n)$ are the representable globular sets.



Example

The spheres $\mathbb{S}^n = \mathbb{D}^{n+1} \setminus \{\text{id}_{n+1}\}$ are their boundaries.



Strict higher categories ii

Definition

A strict ω -category is a globular set X equipped with identity and composition operations for $k < n$:

$$\circ_k: X_n \times_{X_k} X_n \rightarrow X_n$$

$$1_n: X_n \rightarrow X_{n+1}$$

satisfying associativity, unitality and interchange laws.

Strict higher categories ii

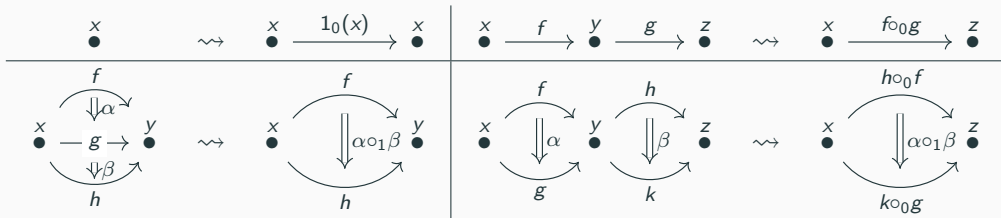
Definition

A strict ω -category is a globular set X equipped with identity and composition operations for $k < n$:

$$\circ_k: X_n \times_{X_k} X_n \rightarrow X_n$$

$$1_n: X_n \rightarrow X_{n+1}$$

satisfying associativity, unitality and interchange laws.



Pasting diagrams

Pasting diagrams are a family of globular sets that admit a unique composite in any strict ω -category. They are generated inductively from $\mathbb{D}^0$ using the **suspension** and **wedge sum** operations:

Pasting diagrams

Pasting diagrams are a family of globular sets that admit a unique composite in any strict ω -category. They are generated inductively from $\mathbb{D}^0$ using the suspension and wedge sum operations:

Definition

The **suspension** of a globular set X has two objects u, v with $\text{Hom}(u, v) = X$ and all other homs empty.

$$x \xrightarrow{f} y \xrightarrow{g} z \quad \rightsquigarrow \quad x \left(\begin{array}{c} \xrightarrow{f} \downarrow \xrightarrow{g} \\ \downarrow y \\ \end{array} \right) z$$

Pasting diagrams

Pasting diagrams are a family of globular sets that admit a unique composite in any strict ω -category. They are generated inductively from $\mathbb{D}^0$ using the suspension and wedge sum operations:

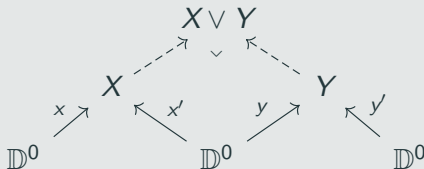
Definition

The suspension of a globular set X has two objects u, v with $\text{Hom}(u, v) = X$ and all other homs empty.

$$x \xrightarrow{f} y \xrightarrow{g} z \rightsquigarrow x \left(\begin{array}{c} \xrightarrow{f} \downarrow \xrightarrow{g} \\ \downarrow y \\ \xrightarrow{\quad} \end{array} \right) z$$

Definition

The **wedge sum** of globular sets X and Y with chosen objects $x, x' \in X_0$ and $y, y' \in Y_0$ is the pushout:



Batanin [3] showed that pasting diagrams are parametrised by rooted, planar trees:

Definition

The set Bat of Batanin trees is defined inductively by $\text{br } L$ being a tree for any list L of Batanin trees.

Batanin [3] showed that pasting diagrams are parametrised by rooted, planar trees:

Definition

The set Bat of **Batanin trees** is defined inductively by $\text{br } L$ being a tree for any list L of Batanin trees. The pasting diagram associated to a tree is defined recursively by

$$\text{Pos}(\text{br}[B_1, \dots, B_n]) = \bigvee_{i=1}^n \Sigma \text{Pos}(B_i)$$

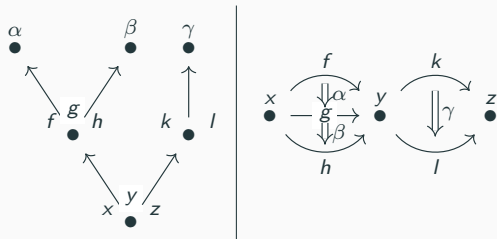
Batanin trees

Batanin [3] showed that pasting diagrams are parametrised by rooted, planar trees:

Definition

The set Bat of Batanin trees is defined inductively by $\text{br } L$ being a tree for any list L of Batanin trees. The **pasting diagram** associated to a tree is defined recursively by

$$\text{Pos}(\text{br}[B_1, \dots, B_n]) = \bigvee_{i=1}^n \Sigma \text{Pos}(B_i)$$



$$B = \text{br}[\text{br}[\text{br}[], \text{br}[]], \text{br}[\text{br}[]]]$$

Towards weak higher categories

Guiding principles

To define weak ω -categories, we replace uniqueness with uniqueness up to equivalence:

Towards weak higher categories

Guiding principles

To define weak ω -categories, we replace uniqueness with uniqueness up to equivalence:

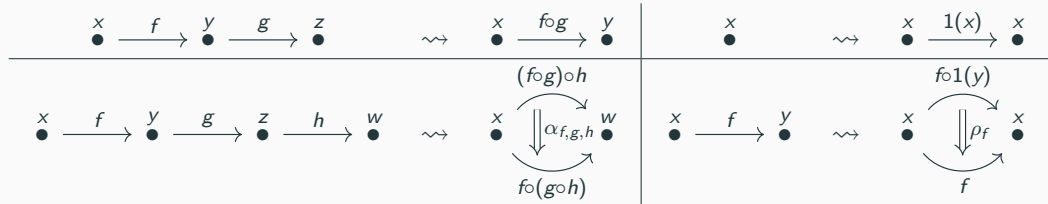
- Parallel ways to compose the source and target of a pasting diagram give rise to a way to compose the entire pasting diagram.
- Parallel composites or coherences over a pasting diagram are related by a coherence operation over the same pasting diagram.

Towards weak higher categories

Guiding principles

To define weak ω -categories, we replace uniqueness with uniqueness up to equivalence:

- Parallel ways to compose the source and target of a pasting diagram give rise to a way to compose the entire pasting diagram.
- Parallel composites or coherences over a pasting diagram are related by a coherence operation over the same pasting diagram.

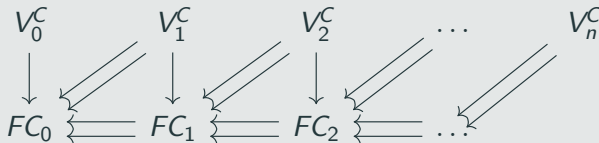


Computads for higher categories

Street [15] defined computads as ways to generate 2-categories. Burroni [8] generalised them to generate strict ω -categories. Batanin [2] further generalised them to generate algebras for finitary monads on globular sets, including strict and weak ω -categories.

Idea

An n -computad is a diagram of sets of the following form



where FC_k is the T -algebra generated by the k -computad $(V_0^C, \dots, V_k^C)$.

Inductive framework i

We will describe the category of computads and an adjunction that induces the *free weak ω -category* monad on globular sets:

$$\text{Glob} \begin{array}{c} \xrightarrow{\quad} \\ \xleftarrow[\text{Cell}]{\perp} \end{array} \text{Comp}$$

Inductive framework i

We will describe the category of computads and an adjunction that induces the *free weak ω -category* monad on globular sets:

$$\text{Glob} \begin{array}{c} \xrightarrow{\quad} \\ \xleftarrow[\text{Cell}]{\perp} \end{array} \text{Comp}$$

Our computads will have some generators marked invertible and their morphisms will send generators to arbitrary cells. This will give the desired morphism of adjunctions:

$$\begin{array}{ccc} \text{Comp} & \xrightarrow{\quad K \quad} & \text{Cat}_\omega \\ & \searrow \text{Cell} \quad \nearrow F & \\ & \text{Glob} & \end{array} \quad \begin{array}{c} \nearrow \gamma \\ \nwarrow U \end{array}$$

Inductive framework ii

We will define by induction on the dimension $-1 \leq n \leq \omega$ the following:

1. a category Comp_n of computads with invertible generators

Inductive framework ii

We will define by induction on the dimension $-1 \leq n \leq \omega$ the following:

1. a category Comp_n of computads with invertible generators
2. a truncation functor $\text{tr}_n: \text{Comp}_{n+1} \rightarrow \text{Comp}_n$

Inductive framework ii

We will define by induction on the dimension $-1 \leq n \leq \omega$ the following:

1. a category Comp_n of computads with invertible generators
2. a truncation functor $\text{tr}_n: \text{Comp}_{n+1} \rightarrow \text{Comp}_n$
3. an inclusion functor $\text{Glob} \rightarrow \text{Comp}_n$

Inductive framework ii

We will define by induction on the dimension $-1 \leq n \leq \omega$ the following:

1. a category Comp_n of computads with invertible generators
2. a truncation functor $\text{tr}_n: \text{Comp}_{n+1} \rightarrow \text{Comp}_n$
3. an inclusion functor $\text{Glob} \rightarrow \text{Comp}_n$
4. a functor $\text{Cell}_n: \text{Comp}_n \rightarrow \text{Glob}_n$

Inductive framework ii

We will define by induction on the dimension $-1 \leq n \leq \omega$ the following:

1. a category Comp_n of computads with invertible generators
2. a truncation functor $\text{tr}_n: \text{Comp}_{n+1} \rightarrow \text{Comp}_n$
3. an inclusion functor $\text{Glob} \rightarrow \text{Comp}_n$
4. a functor $\text{Cell}_n: \text{Comp}_n \rightarrow \text{Glob}_n$
5. A subset $\text{Full}_n(B) \subseteq \text{Glob}(\mathbb{S}^n, \text{Cell}_n \text{Pos } B)$ for each Batanin tree B

Inductive framework ii

We will define by induction on the dimension $-1 \leq n \leq \omega$ the following:

1. a category Comp_n of computads with invertible generators
2. a truncation functor $\text{tr}_n: \text{Comp}_{n+1} \rightarrow \text{Comp}_n$
3. an inclusion functor $\text{Glob} \rightarrow \text{Comp}_n$
4. a functor $\text{Cell}_n: \text{Comp}_n \rightarrow \text{Glob}_n$
5. A subset $\text{Full}_n(B) \subseteq \text{Glob}(\mathbb{S}^n, \text{Cell}_n \text{Pos } B)$ for each Batanin tree B
6. a functor $\text{Mark}_n: \text{Comp}_n \rightarrow \text{Set}$

Inductive framework ii

We will define by induction on the dimension $-1 \leq n \leq \omega$ the following:

1. a category Comp_n of computads with invertible generators
2. a truncation functor $\text{tr}_n: \text{Comp}_{n+1} \rightarrow \text{Comp}_n$
3. an inclusion functor $\text{Glob} \rightarrow \text{Comp}_n$
4. a functor $\text{Cell}_n: \text{Comp}_n \rightarrow \text{Glob}_n$
5. A subset $\text{Full}_n(B) \subseteq \text{Glob}(\mathbb{S}^n, \text{Cell}_n \text{Pos } B)$ for each Batanin tree B
6. a functor $\text{Mark}_n: \text{Comp}_n \rightarrow \text{Set}$
7. a natural transformations $\text{fwd}, \text{linv}, \text{rinv}: \text{Mark}_n \Rightarrow \text{Cell}_n$

Inductive framework ii

We will define by induction on the dimension $-1 \leq n \leq \omega$ the following:

1. a category Comp_n of computads with invertible generators
2. a truncation functor $\text{tr}_n: \text{Comp}_{n+1} \rightarrow \text{Comp}_n$
3. an inclusion functor $\text{Glob} \rightarrow \text{Comp}_n$
4. a functor $\text{Cell}_n: \text{Comp}_n \rightarrow \text{Glob}_n$
5. A subset $\text{Full}_n(B) \subseteq \text{Glob}(\mathbb{S}^n, \text{Cell}_n \text{Pos } B)$ for each Batanin tree B
6. a functor $\text{Mark}_n: \text{Comp}_n \rightarrow \text{Set}$
7. a natural transformations $\text{fwd}, \text{linv}, \text{rinv}: \text{Mark}_n \Rightarrow \text{Cell}_n$

Inductive framework ii

We will define by induction on the dimension $-1 \leq n \leq \omega$ the following:

1. a category Comp_n of computads with invertible generators
2. a truncation functor $\text{tr}_n: \text{Comp}_{n+1} \rightarrow \text{Comp}_n$
3. an inclusion functor $\text{Glob} \rightarrow \text{Comp}_n$
4. a functor $\text{Cell}_n: \text{Comp}_n \rightarrow \text{Glob}_n$
5. A subset $\text{Full}_n(B) \subseteq \text{Glob}(\mathbb{S}^n, \text{Cell}_n \text{Pos } B)$ for each Batanin tree B
6. a functor $\text{Mark}_n: \text{Comp}_n \rightarrow \text{Set}$
7. a natural transformations $\text{fwd}, \text{linv}, \text{rinv}: \text{Mark}_n \Rightarrow \text{Cell}_n$

starting from $\text{Comp}_{-1} = \mathbb{1}$ and $\text{Cell}_{-1} = \text{Mark}_{-1} = \emptyset$.

Computads and their morphisms

Definition

An $(n+1)$ -computad C with invertible generators consists of:

- an n -computad $\text{tr}_n C$,
- two sets V_{n+1}^C and U_{n+1}^C of ordinary and invertible generators respectively,
- an attaching function $\phi_n^C: V_{n+1}^C + U_{n+1}^C \rightarrow \text{Glob}(\mathbb{S}^n, \text{Cell}_n \text{tr}_n C)$

Definition

A morphism $f: C \rightarrow D$ of $(n+1)$ -computads consists of:

- a morphism $\text{tr}_n f: \text{tr}_n C \rightarrow \text{tr}_n D$
- a function $f_V: V_{n+1}^C \rightarrow \text{Cell}_{n+1}(D)$
- a function $f_U: U_{n+1}^C \rightarrow \text{Mark}_{n+1}(D)$

preserving the source and target of each ordinary and invertible generator.

Definition

The set $\text{Mark}_{n+1}(C)$ is inductively generated by:

- $\text{igen}(u)$ for $u \in U_{n+1}^C$
- $\text{lunit}(i)$ and $\text{runit}(i)$ for $i \in \text{Mark}_n(\text{tr}_n C)$

Cells and their boundaries

Definition

The set $\text{Mark}_{n+1}(C)$ is inductively generated by:

- $\text{igen}(u)$ for $u \in U_{n+1}^C$
- $\text{lunit}(i)$ and $\text{runit}(i)$ for $i \in \text{Mark}_n(\text{tr}_n C)$

Definition

The set $\text{Cell}_{n+1}(C)$ is inductively generated by:

- $\text{gen}(v)$ for $v \in V_{n+1}^C$
- $\text{fwd}(i)$, $\text{linv}(i)$ and $\text{rinv}(i)$ for $i \in \text{Mark}_{n+1}(C)$
- $\text{coh}_{B,A}[f]$ for
 - Batanin tree B with $\dim B \leq n+1$
 - $A \in \text{Full}_n(B)$
 - $f: \text{Pos}(B) \rightarrow C$

Cells and their boundaries

Definition

The set $\text{Mark}_{n+1}(C)$ is inductively generated by:

- $\text{igen}(u)$ for $u \in U_{n+1}^C$
- $\text{lunit}(i)$ and $\text{runit}(i)$ for $i \in \text{Mark}_n(\text{tr}_n C)$

Definition

The set $\text{Cell}_{n+1}(C)$ is inductively generated by:

- $\text{gen}(v)$ for $v \in V_{n+1}^C$
- $\text{fwd}(i)$, $\text{linv}(i)$ and $\text{rinv}(i)$ for $i \in \text{Mark}_{n+1}(C)$
- $\text{coh}_{B,A}[f]$ for
 - Batanin tree B with $\dim B \leq n+1$
 - $A \in \text{Full}_n(B)$
 - $f: \text{Pos}(B) \rightarrow C$

$\text{ibdry}: \text{Mark}_{n+1}(C) \rightarrow \text{Glob}(\mathbb{S}^n, \text{Cell}_n \text{tr}_n C)$

$\text{igen}(u) \mapsto \phi_{n+1}^C(u)$

$\text{lunit}(i) \mapsto (\text{linv}(i) \circ \text{fwd}(i), 1)$

$\text{runit}(i) \mapsto (1, \text{fwd}(i) \circ \text{rinv}(i))$

$\text{bdry}: \text{Cell}_{n+1}(C) \rightarrow \text{Glob}(\mathbb{S}^n, \text{Cell}_n \text{tr}_n C)$

$\text{gen}(v) \mapsto \phi_{n+1}^C(v)$

$\text{fwd}(i) \mapsto \text{ibdry}(i)$

$\text{linv}(i) \mapsto \text{ibdry}(i)^{\text{op}}$

$\text{rinv}(i) \mapsto \text{ibdry}(i)^{\text{op}}$

$\text{coh}_{B,A}[f] \mapsto (\text{tr}_n f)_*(A)$

Full spheres

Full spheres are an analogue of the admissible pairs of maps in the sense of Maltsiniotis [14]. They are the boundaries of the operations of ω -categories:

Definition

The **support** of a cell $c \in \text{Cell}_n(C)$ is the set of generators that appear in c or its boundary. A sphere $A = (a, b)$ of $\text{Cptd}_{n+1} \text{Pos}(B)$ is **full** when

- the support of a contains all positions of $\partial_n^- B$
- the support of b contains all positions of $\partial_n^+ B$

Full spheres

Full spheres are an analogue of the admissible pairs of maps in the sense of Maltsiniotis [14]. They are the boundaries of the operations of ω -categories:

$$\begin{aligned} B_1 &= \bullet \xrightarrow{x} \bullet \xrightarrow{f} \bullet \xrightarrow{y} \bullet \xrightarrow{g} \bullet \xrightarrow{z} \bullet \\ A_1 &= (x, z) \quad \text{full} \\ A_2 &= (z, x) \quad \text{not full} \end{aligned}$$

Definition

The **support** of a cell $c \in \text{Cell}_n(C)$ is the set of generators that appear in c or its boundary. A sphere $A = (a, b)$ of $\text{Cptd}_{n+1} \text{Pos}(B)$ is **full** when

- the support of a contains all positions of $\partial_n^- B$
- the support of b contains all positions of $\partial_n^+ B$

Full spheres

Full spheres are an analogue of the admissible pairs of maps in the sense of Maltsiniotis [14]. They are the boundaries of the operations of ω -categories:

Definition

The **support** of a cell $c \in \text{Cell}_n(C)$ is the set of generators that appear in c or its boundary. A sphere $A = (a, b)$ of $\text{Cptd}_{n+1} \text{Pos}(B)$ is **full** when

- the support of a contains all positions of $\partial_n^- B$
- the support of b contains all positions of $\partial_n^+ B$

$$B_1 = \begin{array}{ccccc} x & & f & & y & & g & & z \\ \bullet & \longrightarrow & & \longrightarrow & \bullet & \longrightarrow & & \longrightarrow & \bullet \end{array}$$

$$A_1 = (x, z) \quad \text{full}$$

$$A_2 = (z, x) \quad \text{not full}$$

$$B_2 = \begin{array}{ccccccc} x & & f & & y & & g & & z & & h & & w \\ \bullet & \longrightarrow & & \longrightarrow & \bullet & \longrightarrow & & \longrightarrow & \bullet & \longrightarrow & & \longrightarrow & \bullet \end{array}$$

$$A_3 = (x, w) \quad \text{full}$$

Full spheres

Full spheres are an analogue of the admissible pairs of maps in the sense of Maltsiniotis [14]. They are the boundaries of the operations of ω -categories:

Definition

The **support** of a cell $c \in \text{Cell}_n(C)$ is the set of generators that appear in c or its boundary. A sphere $A = (a, b)$ of $\text{Cptd}_{n+1} \text{Pos}(B)$ is **full** when

- the support of a contains all positions of $\partial_n^- B$
- the support of b contains all positions of $\partial_n^+ B$

$$B_1 = \begin{array}{c} x \quad \quad y \quad \quad z \\ \bullet \xrightarrow{f} \bullet \xrightarrow{g} \bullet \end{array}$$

$$A_1 = (x, z) \quad \text{full}$$

$$A_2 = (z, x) \quad \text{not full}$$

$$B_2 = \begin{array}{c} x \quad \quad y \quad \quad z \quad \quad w \\ \bullet \xrightarrow{f} \bullet \xrightarrow{g} \bullet \xrightarrow{h} \bullet \end{array}$$

$$A_3 = (x, w) \quad \text{full}$$

$$A_4 = (f \circ g \circ h, (f \circ g) \circ h) \quad \text{full}$$

$$A_5 = ((f \circ g) \circ h, f \circ g \circ h) \quad \text{full}$$

$$\begin{aligned} f \circ g &= \text{coh}_{B_1, A_1}[f \mapsto f, g \mapsto g] \\ (f \circ g) \circ h &= \text{coh}_{B_1, A_1}[f \mapsto f \circ g, g \mapsto h] \\ f \circ g \circ h &= \text{coh}_{B_2, A_3}[\text{id}] \end{aligned}$$

Definition

The category Comp of **computads with invertible generators** is the limit:

$$\dots \xrightarrow{\text{tr}_n} \text{Comp}_n \xrightarrow{\text{tr}_{n-1}} \dots \xrightarrow{\text{tr}_1} \text{Comp}_1 \xrightarrow{\text{tr}_0} \text{Comp}_0$$

Definition

The category Comp of computads with invertible generators is the limit:

$$\dots \xrightarrow{\text{tr}_n} \text{Comp}_n \xrightarrow{\text{tr}_{n-1}} \dots \xrightarrow{\text{tr}_1} \text{Comp}_1 \xrightarrow{\text{tr}_0} \text{Comp}_0$$

The remaining data gives rise to an adjunction:

$$\text{Glob} \begin{array}{c} \xrightarrow{\quad} \\ \perp \\ \xleftarrow{\text{Cell}} \end{array} \text{Comp}$$

and hence a monad T on globular sets.

Computads and ω -categories

Definition

The category Comp of computads with invertible generators is the limit:

$$\dots \xrightarrow{\text{tr}_n} \text{Comp}_n \xrightarrow{\text{tr}_{n-1}} \dots \xrightarrow{\text{tr}_1} \text{Comp}_1 \xrightarrow{\text{tr}_0} \text{Comp}_0$$

The remaining data gives rise to an adjunction:

$$\text{Glob} \begin{array}{c} \xrightarrow{\quad \perp \quad} \\ \xleftarrow{\quad \text{Cell} \quad} \end{array} \text{Comp}$$

and hence a monad T on globular sets.

Theorem

The category of *weak ω -categories* is the category of T -algebras.

$$\begin{array}{ccc} \text{Comp} & \xrightarrow{\quad K \quad} & \text{Cat}_\omega \\ & \searrow \text{Cell} & \nearrow F \\ & \text{Glob} & \nearrow U \end{array}$$

Variants of computads

Definition

An **ordinary computad** is a computad with no invertible generators.

Definition

A **rigid morphism** $f: C \rightarrow D$ is one that sends generators to generators.

Variants of computads

Definition

An **ordinary computad** is a computad with no invertible generators.

Definition

A **rigid morphism** $f: C \rightarrow D$ is one that sends generators to generators.

$$\begin{array}{ccc} \text{Comp}^{\text{ord,rig}} & \longrightarrow & \text{Comp}^{\text{ord}} \\ \downarrow & \lrcorner & \downarrow \\ \text{Comp}^{\text{rig}} & \longrightarrow & \text{Comp} \end{array}$$

Variants of computads

Definition

An **ordinary computad** is a computad with no invertible generators.

Definition

A **rigid morphism** $f: C \rightarrow D$ is one that sends generators to generators.

$$\begin{array}{ccc} \text{Comp}^{\text{ord,rig}} & \longrightarrow & \text{Comp}^{\text{ord}} \\ \downarrow & \lrcorner & \downarrow \\ \text{Comp}^{\text{rig}} & \longrightarrow & \text{Comp} \end{array}$$

Theorem

The rigid subcategories are presheaf topoi.

Remark

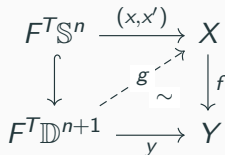
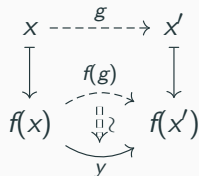
This fails in the strict case!

Weak equivalences and trivial fibrations

Definition

A **weak equivalence** $f: X \rightarrow Y$ between ω -categories is a morphism satisfying that:

- for every $y \in Y_0$ there exists $x \in X_0$ and $f(x) \xrightarrow{\sim} y$
- for every $x, x' \in X_0$ and $y: f(x) \rightarrow f(x')$, there exists $g: x \rightarrow x'$ and $f(g) \xrightarrow{\sim} y$



Definition

A **trivial fibration** is a morphism that has the right lifting property with respect to $F^T \mathbb{S}^n \hookrightarrow F^T \mathbb{D}^{n+1}$ for all $n \geq -1$.

Every ω -category is free!

Theorem

Every ω -category is weakly equivalent to one generated by a computad.

Idea of proof.

- The inclusion $\text{Comp}^{\text{ord}, \text{rig}} \hookrightarrow \text{Comp} \xrightarrow{K} \text{Cat}_\omega$ admits a right adjoint Q .
- The counit $KQX \rightarrow X$ is a trivial fibration and hence a weak equivalence. □

Every ω -category is free!

Theorem

Every ω -category is weakly equivalent to one generated by a computad.

Idea of proof.

- The inclusion $\text{Comp}^{\text{ord,rig}} \hookrightarrow \text{Comp} \xrightarrow{K} \text{Cat}_\omega$ admits a right adjoint Q .
- The counit $KQX \rightarrow X$ is a trivial fibration and hence a weak equivalence. □

Remark

Computads are the cofibrant objects for the factorisation system generated by the sphere to disk inclusions.

Did we need invertibility?

Proposition

The inclusion $\text{Comp}^{\text{ord}} \hookrightarrow \text{Comp}$ admits a right adjoint R that preserves the generated ω -category.

Idea of proof.

We construct the computad RC by induction on the dimension. Each element of $\text{Mark}_n(C)$ gives rise to three generators of RC , one for its itself and two for its inverses. □

Nonetheless, we get a very succinct description of certain weak ω -categories by allowing invertible generators. For example, we can define the walking isomorphism $\mathbb{E}^{n+1}$ to be the computad extending $\mathbb{S}^n$ with a single invertible generator.

Future directions and consequences

- This work is part of our pursuit of a model structure on Cat_ω .

Future directions and consequences

- This work is part of our pursuit of a model structure on Cat_ω .
- What remaining to be done is showing that certain cylinder objects exist.

Future directions and consequences

- This work is part of our pursuit of a model structure on Cat_ω .
- What remaining to be done is showing that certain cylinder objects exist.
- This is a stepping stone towards comparing (marked) ω -categories with other models of (∞, ∞) -categories, such as Θ -spaces, or complicial sets.

Future directions and consequences

- This work is part of our pursuit of a model structure on Cat_ω .
- What remaining to be done is showing that certain cylinder objects exist.
- This is a stepping stone towards comparing (marked) ω -categories with other models of (∞, ∞) -categories, such as Θ -spaces, or complicial sets.
- Extending the theory to ω -groupoids would yield a new model for homotopy types, giving new insight on their algebraic structure and strengthening their connection to intensional type theory.

Thank you for your attention!

References i

- [1] Dimitri Ara. “Sur les ∞ -groupoïdes de Grothendieck et une variante ∞ -catégorique”. Thèse de doctorat. 2010.
- [2] Michael A. Batanin. *Computads and slices of operads*. 2002. Pre-published.
- [3] Michael A. Batanin. “Monoidal Globular Categories As a Natural Environment for the Theory of Weak n -Categories”. In: *Advances in Mathematics* 136.1 (1998).
- [4] Thibaut Benjamin, Camil Champin, and Ioannis M. *Computads with Invertible Generators for Weak ω -Categories*. 2026. Pre-published.
- [5] Thibaut Benjamin, Ioannis M., and Chiara Sarti. “CaTT contexts are finite computads”. In: *Proceedings of the 40th Conference on Mathematical Foundations of Programming Semantics*. 2024.
- [6] Thibaut Benjamin, Samuel Mimram, and Eric Finster. “Globular weak ω -categories as models of a type theory”. In: *Higher Structures* 2 (2024), pp. 1–69.
- [7] John Bourke. “Iterated algebraic injectivity and the faithfulness conjecture”. In: *Higher Structures* 4.2 (2020).
- [8] Albert Burroni. “Higher-dimensional word problems with applications to equational logic”. In: *Theoretical Computer Science* 115.1 (1993), pp. 43–62.
- [9] Christopher J. Dean et al. “Computads for weak ω -categories as an inductive type”. In: *Advances in Mathematics* 450 (2024).

- [10] Eric Finster and Samuel Mimram. “A type-theoretical definition of weak ω -categories”. In: *Proceedings of the 32nd Annual ACM/IEEE Symposium on Logic in Computer Science*. 2017.
- [11] Alexander Grothendieck. *Pursuing Stacks*. 1983.
- [12] Yves Lafont, François Métayer, and Krzysztof Worytkiewicz. “A folk model structure on omega-cat”. In: *Advances in Mathematics* 224.3 (2010).
- [13] Tom Leinster. *Higher operads, higher categories*. 298. Cambridge University Press, 2004.
- [14] Georges Maltsiniotis. *Grothendieck ∞ -groupoids, and still another definition of ∞ -categories*. 2010.
- [15] Ross Street. “Limits indexed by category-valued 2-functors”. In: *Journal of Pure and Applied Algebra* 8.2 (1976).

Bonus: The grammar of CaTT

We can alternatively reason about weak ω -categories and coinductive invertibility using dependent type theory. This allows us to construct proof assistants for reasoning about weak ω -categories.

Bonus: The grammar of CaTT

We can alternatively reason about weak ω -categories and coinductive invertibility using dependent type theory. This allows us to construct proof assistants for reasoning about weak ω -categories. The grammar of the type theory CaTT:

Variables	x, y	$::=$	$x \in \mathbb{N}$	
Terms	t, u	$::=$	x	$ \text{coh}_{\Delta, A}[\sigma]$
Types	A, B	$::=$	$\star$	$ u \rightarrow_A v.$
Contexts	Γ, Δ	$::=$	$\emptyset$	$ \Gamma, x: A$
Substitutions	σ, τ	$::=$	$\langle \rangle$	$ \langle \sigma, x \mapsto t \rangle$

Bonus: The grammar of CaTT

We can alternatively reason about weak ω -categories and coinductive invertibility using dependent type theory. This allows us to construct proof assistants for reasoning about weak ω -categories. The grammar of the type theory CaTT:

Variables	$x, y ::= x \in \mathbb{N}$	
Terms	$t, u ::= x$	$ \text{coh}_{\Delta, A}[\sigma]$
Types	$A, B ::= \star$	$ u \rightarrow_A v.$
Contexts	$\Gamma, \Delta ::= \emptyset$	$ \Gamma, x: A$
Substitutions	$\sigma, \tau ::= \langle \rangle$	$ \langle \sigma, x \mapsto t \rangle$

Its judgement forms are the usual ones for dependent type theory:

$$\Gamma \vdash \qquad \Gamma \vdash A \qquad \Gamma \vdash t: A \qquad \Gamma \vdash \sigma: \Delta$$

with three additional auxiliary judgement forms:

$$\Gamma \vdash_{\text{ps}} \qquad \Gamma \vdash_{\text{ps}} x: A \qquad \Gamma \vdash_{\text{full}} A$$

The rules of CaTT

The rules of CaTT contain the usual structural rules for contexts, substitutions and variables. Its type and term formation rules are the following ones:

$$\frac{\Gamma \vdash}{\Gamma \vdash \star} \quad \frac{\Gamma \vdash u : A \quad \Gamma \vdash v : A}{\Gamma \vdash u \rightarrow_A v} \quad \frac{\Gamma \vdash_{\text{ps}} \quad \Gamma \vdash_{\text{full}} A \quad \Delta \vdash \delta : \Gamma}{\Delta \vdash \text{coh}_{\Gamma, A}[\sigma] : A[\sigma]}$$

The rules for pasting contexts and full types are more involved, but analogous to the ones described in the setting of computads.

Theorem

Contexts of CaTT are finite computads and models of CaTT are weak ω -categories.

An extension of CaTT

We may extend CaTT with a new type former of invertible terms and six destructors:

$$\frac{\Gamma \vdash f: x \rightarrow_A y}{\Gamma \vdash \text{Inv}(f)} \quad \frac{\Gamma \vdash i: \text{Inv}_A(f)}{\begin{array}{ll} \Gamma \vdash \text{linv}(i) : A^{\text{op}} & \Gamma \vdash \text{rinv}(i) : A^{\text{op}} \\ \Gamma \vdash \text{lunit}(i) : \text{linv}(i) * f \rightarrow \text{id} & \Gamma \vdash \text{runit}(i) : \text{id} \rightarrow f * \text{rinv}(i) \\ \Gamma \vdash \text{ilunit}(i) : \text{Inv}(\text{lunit}(i)) & \Gamma \vdash \text{irunit}(i) : \text{Inv}(\text{runit}(i)) \end{array}}$$

We also add a constructor and a recursion principle as well as β and η rules.

$$\begin{array}{ccc} S_{CaTT} & \hookrightarrow & S_{ICaTT} \\ \downarrow & & \downarrow \\ \text{Mod}(CaTT) & \xrightleftharpoons{\perp} & \text{Mod}(ICaTT) \end{array}$$

Theorem

The extension of CaTT with invertible terms is conservative.

$$\begin{array}{ccc} S_{CaTT} & \hookrightarrow & S_{ICaTT} \\ \downarrow & & \downarrow \\ \text{Mod}(CaTT) & \xrightleftharpoons{\perp} & \text{Mod}(ICaTT) \end{array}$$

$$\begin{array}{ccc} S_{CaTT} & \hookrightarrow & S_{ICaTT} \\ \downarrow & & \downarrow \\ \text{Mod}(CaTT) & \xrightleftharpoons{\perp} & \text{Mod}(ICaTT) \end{array}$$

Theorem

The extension of $CaTT$ with invertible terms is conservative.

Remark

Via the adjunction on the left we can describe some infinite computads as contexts of the extended type theory.

$$\begin{array}{ccc} S_{CaTT} & \hookrightarrow & S_{ICaTT} \\ \downarrow & & \downarrow \\ \text{Mod}(CaTT) & \xrightleftharpoons{\perp} & \text{Mod}(ICaTT) \end{array}$$

Theorem

The extension of CaTT with invertible terms is conservative.

Remark

Via the adjunction on the left we can describe some infinite computads as contexts of the extended type theory.

Theorem

We have semantics in fibrant marked weak ω -categories.